# Lab03

**Task 1 (2 points)**:

Create your GitHub repository, add a file to the repository, and add **Jonathan-Skeete** as a collaborator by following the instructions below.

1. Go to "github.com" and create an account or sign in to your existing account.
2. Once on the Dashboard page, click on the green button labeled **"New"** to create a new repository.
3. On the New Repository page, name your repository **"CS281"** and make it **PRIVATE**. Afterward, click the green button labeled **"Create repository"**.
4. On your Repository page, click the **"create a new file"** link at the bottom of the blue box.
5. On the new page, type **"info.txt"** in the textbox with the content **"Name your file..."**. Afterward, write your name, email address, and GitHub URL (e.g. https://github.com/username/CS281) each on a separate line in the main textbox. Last, click on the green button labeled **"Commit changes..."** and complete any additional prompts.
6. Back on your Repository page, select the option **Settings** from the top menu.
7. On the new page, select the option **Collaborators** under the **Access** tab from the side menu; and then, click on the green button labeled **"Add people"** in the box titled **"Manage access"**.
8. Find **Jonathan-Skeete** in the search bar of the new window, and then, click the green button.

**Task 2 (2 points)**:

Create a Neon account. Neon will be used to host a cloud-based PostgreSQL database that your Python programs can connect to and interact with.

1. Go to “neon.com” and sign up or log in to youyr existing account.
2. Once your free account is created, you will be prompted to “**Create project**”. Under **“Project name”** enter “CS281” then click **“Create”**. If prompted to “Get started with Neon + AI”, ignore and exit the prompt.
3. Once on the Dashboard page, click on the button labeled **"Connect"** towards the top right of the screen. Click the drop-down menu under **“Database”** then click **“Create new database”**. Under **“Database name”** type “university” then click **“Create”**. Ensure “university” now appears under **“Database”**. At the bottom of the window, click **“Show password”** then **“Copy snippet”**. (You’ve just copied the connection string.)

**Task 3 (2 points)**:

Open your GitHub repository in Codespace.

1. In the terminal run the command: `pip install -r requirements.txt`
2. In the terminal run the command: `cd Labs/Lab03/`
3. In the terminal run the command: `cp University/.env.example University/.env`
4. In the .env file, paste the copied connection string from task 2 after: **`DB_CONNECTION_STRING=`**
5. You’ll now need to parse the connection string to get the **`DB_HOST`** and **`DB_PASSWORD`** values.
   Example Connection String:
   `postgresql://neondb_owner:password@ep-example-123456-pooler.us-east-1.aws.neon.tech/university?sslmode=require&channel_binding=require`
6. The **`host`** can be found between **`@`** and the first **`/`** after **`@`**. The **`password`** can be found between the first **`:`** before **`@`** and **`@`** in the connection string.
   ```
   DB_HOST=ep-example-123456-pooler.us-east-1.aws.neon.tech
   DB_PASSWORD=password
   ```
7. Run **test.py** to ensure a successful connection to the university database.

**Task 4 (2 points)**:

Inside **Labs/Lab03/University/**:

1. Create a file called **ddl.sql** that will hold the create table statements for each relation. Use the DDL from the online SQL interpreter or pages 1288 through 1291 of the textbook.
2. All statements should end with a semicolon.
3. Replace `CREATE TABLE` with `CREATE TABLE IF NOT EXISTS`.
4. Run **setup.py**.

**Task 5 (2 points)**:

Inside **Labs/Lab03/University/**, create a new folder called **DATA** that will hold CSV files for each relation.

1. Create a file called **classroom.csv**. Add all classroom records to the CSV file with the first line being the column names. All values should be separated by commas.
2. Create a file called **department.csv**. Add all department records to the CSV file with the first line being the column names. All values should be separated by commas.
3. Create a file called **course.csv**. Add all course records to the CSV file with the first line being the column names. All values should be separated by commas.
4. Create a file called **instructor.csv**. Add all instructor records to the CSV file with the first line being the column names. All values should be separated by commas.
5. Create a file called **section.csv**. Add all section records to the CSV file with the first line being the column names. All values should be separated by commas.
6. Create a file called **teaches.csv**. Add all teaches records to the CSV file with the first line being the column names. All values should be separated by commas.
7. Create a file called **student.csv**. Add all student records to the CSV file with the first line being the column names. All values should be separated by commas.
8. Create a file called **takes.csv**. Add all takes records to the CSV file with the first line being the column names. All values should be separated by commas.
9. Create a file called **advisor.csv**. Add all advisor records to the CSV file with the first line being the column names. All values should be separated by commas.
10. Create a file called **time_slot.csv**. Add all time_slot records to the CSV file with the first line being the column names. All values should be separated by commas.
11. Create a file called **prereq.csv**. Add all prereq records to the CSV file with the first line being the column names. All values should be separated by commas.
12. Run **load.py**.

Run **query.py** to test/execute queries.

Push your work to GitHub using the following commands:

```
git pull
git add .
git commit –m “lab03 upload”
git push
```